

Hochschule Ravensburg-Weingarten
University of Applied Sciences



Specification of

RISC-V 64-Bit Single-Core Microcontroller

Author: Yash Balu Sangale (29346383)

Co-Author: Neel Karia (16346352)

Study Program: M.Eng. Electrical Engineering and Embedded
Systems

Faculty of Electrical Engineering and Computer Science

Supervisor: Prof. Dr.-Ing. A. Siggelkow

Revision: 1.1

Date: June 23, 2026

Contents

1	Introduction	1
1.1	Purpose of Document	1
1.2	Scope	1
1.3	Reference Documents	1
2	Product Overview	2
2.1	Top Level Description	2
2.1.1	Introduction	3
2.1.2	Key Features	3
2.2	Architecture Concepts: Overview	4
2.2.1	Functional Block Diagram	4
2.2.2	Package	4
2.3	System View	4
2.3.1	Application System Description and Use Cases	4
2.3.2	System Description	5
2.3.3	System Integration and Application Circuit	5
2.3.4	Compliances	6
2.4	Architecture Concepts Overview	6
2.4.1	Technology	6
2.4.2	System Memory Concept	7
2.4.3	Software Architecture	7
2.4.4	Interprocessor Communication Concept	8
2.5	Functional Block Overview	8
2.6	Pin List	9
3	Architecture Concepts	10
3.1	Technology	11
3.2	CPU Concept	11
3.3	Memory Concept	12
3.3.1	Memory Map	13
3.4	Software Architecture Concept	13
3.5	Interprocessor Communication Concept	13
3.6	Interrupt Concept	14
3.7	Bus Concept	14
3.7.1	Wishbone Protocol Specification	15
3.8	System Control Concept	15
3.9	Register Concept	16
3.10	Clock Concept	16

3.11	Power Concept	17
3.12	Power Concept	17
3.13	System Protection and Security Concept	17
3.14	Debug Concept	17
3.15	Test and Debug Concept	17
3.16	Attacks, Hazards and Mitigation Methodologies	18
3.16.1	Meltdown and Spectre	18
4	Description of the Design Elements	21
4.1	Central Processing Unit (CPU) Core	21
4.1.1	HW Interfaces to Other Blocks	21
4.1.2	Bus Interface	22
4.1.3	Registers	22
4.1.4	Functional Overview	22
4.1.5	Structural Overview	23
4.1.6	Interrupts	23
4.1.7	Clock and Reset	23
4.1.8	Voltage Class	23
4.2	Floating-Point Unit (FPU)	23
4.2.1	Introduction	23
4.2.2	Compliance and Standards	24
4.2.3	Floating-Point Data Formats	24
4.2.4	Floating-Point Register File	25
4.2.5	Rounding Modes	25
4.2.6	Exception Handling	26
4.2.7	Supported Instructions	26
4.2.8	FPU Microarchitecture	27
4.2.9	Hardware Interfaces	29
4.2.10	Floating-Point CSR Registers	30
4.2.11	Address Map and Integration	30
4.2.12	Power Considerations	30
4.2.13	Integration with top_npl	30
4.2.14	Verification and Test Strategy	31
4.2.15	Future Extensions	31
5	Validation and Test	33
6	Toolchain	34
6.1	Introduction of the Toolchain	34
6.2	Technologies Used	34
6.2.1	Build and Automation Technologies	35
6.2.2	Simulation and Verification Tools	35
6.2.3	Development and Integration Tools	35
6.3	Usage of the Toolchain	35
6.3.1	Terminal-Based Usage	35
6.3.2	VS Code Extension Usage	36
6.4	Further Development of the Toolchain	37
6.4.1	CMake Enhancements	37
6.4.2	VS Code Extension Enhancements	37

7	Test And Debug	38
8	Attacks, Hazards and Mitigation Methodologies	39
8.1	Meltdown and Spectre	39
8.1.1	Background and Threat Model	39
8.1.2	Architectural Preconditions	39
8.1.3	Meltdown (CVE-2017-5754)	40
8.1.4	Spectre (CVE-2017-5753 / CVE-2017-5715)	41
8.1.5	Cache Timing Side-Channel	43
8.1.6	Countermeasures	43
8.1.7	Vulnerability Summary	45
9	Electrical Characteristics	46
9.1	Operating Conditions	46

List of Figures

2.1	Top Level Block Diagram of RISC-V 64-bit Base Integer Instruction Set (RV64I) System-on-Chip	2
3.1	Pulse-based enable logic for synchronous control signal generation	10
3.2	Conceptual illustration of transient execution and cache-based side-channel leakage	19
4.1	CPU Core interface diagram showing I-Bus, D-Bus, and Wishbone connections to system blocks.	22
4.2	CPU bus architecture — direct Harvard Architecture-style connections to Instruction Memory (I-MEM)/Data Memory (D-MEM) and Wishbone master for peripherals.	22
4.3	CPU Core internal structure showing Instruction Decoder, Arithmetic Logic Unit (ALU), Program Counter, and Register File.	23
4.4	Simplified Harvard Architecture architecture connection of CPU to I-MEM, D-MEM, and Wishbone peripherals.	23
8.1	Meltdown attack execution flow on an OoO processor with virtual memory.	41
8.2	Spectre Variant 1 execution flow: bounds-check bypass via branch misprediction.	42
8.3	Commit-gated cache-fill suppression: cache updates are held until ROB commit, preventing transient execution from producing observable side-effects.	45

List of Tables

1.1	History / Revision – Chapter 1	1
2.1	Revision History – Chapter 2	2
3.1	Revision History – Chapter 3	10
4.1	Revision History – Chapter 4	21
4.2	CPU Core Register Summary	22
4.3	FPU Unit Latency (Current Implementation)	28
4.4	FPU Functional Unit Map	29
4.5	fpu_top Port List	30
5.1	Revision History – Chapter 6	33
6.1	History / Revision – Chapter 5	34
7.1	Revision History – Chapter 5	38
8.1	Revision History – Chapter 8: Attacks, Hazards and Mitigation Methodologies	39
8.2	Meltdown Precondition Mapping for RWU64IM (Extended Configuration)	41
8.3	Spectre Precondition Mapping for RWU64IM (Extended Multi-Core Configuration)	42
8.4	Countermeasure Applicability — RWU64IM Extended Configuration	44
8.5	Transient Execution Vulnerability Assessment — RWU64IM Multi-Core Extension	45
9.1	Revision History – Chapter 7	46
9.2	Operating Conditions	46

Glossary

Harvard Architecture A computer architecture with separate instruction and data memory paths. iv, 3, 4, 6–8, 10, 11, 15–17

pipeline A series of processing stages that execute instructions in overlapping steps. 3–7, 10, 11, 16, 17

System-on-Chip An integrated circuit containing processing cores, memory, and peripherals. iv, 1–3, 6, 9, 15

Acronyms

ALU	Arithmetic Logic Unit. iv, 7, 10, 13, 16, 17
ASIC	Application-Specific Integrated Circuit. 6, 9
CPU	Central Processing Unit. ii–v, 4–17
D-MEM	Data Memory. iv, 3, 5–8, 10–13, 15–17
FPGA	Field-Programmable Gate Array. 4–7, 9
GPIO	General Purpose Input/Output. 3–5, 7, 8, 11, 13–15
HDL	Hardware Description Language. 9
I-MEM	Instruction Memory. iv, 3, 5–8, 10, 11, 13, 15–17
IRQ	Interrupt Request or Interrupt Request Line. 3, 7, 8, 12–14, 17
ISA	Instruction Set Architecture. 1, 3, 5, 7, 10
PC	Program Counter. 10, 13, 16, 17
PWM	Pulse Width Modulation. 3, 5, 8
QSPI	Quad Serial Peripheral Interface. 3–5, 7, 8, 11, 13–15
RISC-V	Reduced Instruction Set Computer - Five. 1, 3, 4, 6, 7, 9, 10, 16
RV64A	RISC-V 64-bit Extension for Atomic Instructions. 3
RV64FD	RISC-V 64-bit Extension for Single + Double-Precision Floating-Point. 3
RV64I	RISC-V 64-bit Base Integer Instruction Set. iv, 2–5, 7, 10, 15, 16
RV64M	RISC-V 64-bit Extension for Integer Multiplication and Division. 3
SRAM	Static Random-Access Memory. 9
UART	Universal Asynchronous Receiver-Transmitter. 3–5, 7, 8, 11, 13–15

Chapter 1

Introduction

Table 1.1: History / Revision – Chapter 1

Version	Date	Author	Remarks
1.0	January 22, 2026	Y. Sangale	Initial template draft
1.0	April 14, 2026	Y. Sangale	FPU Specs draft

1.1 Purpose of Document

This document defines the first specification of the Reduced Instruction Set Computer - Five (RISC-V) single-instruction-bit microcontroller, covering architectural, electrical, and functional aspects. Revision 1.1 adds the Floating-Point Unit (FPU) specification as an additional design element.

1.2 Scope

The specification outlines the design decisions, Instruction Set Architecture (ISA) subset, interfaces, and architectural details of the RISC-V System-on-Chip. This revision extends the scope to include the IEEE 754-2008 compliant double-precision FPU.

1.3 Reference Documents

- RISC-V Unprivileged ISA Specification, Volume I
- RISC-V Privileged Architecture Specification
- IEEE 754-2008: Standard for Floating-Point Arithmetic
- <https://docs.riscv.org/reference/isa/unpriv/f-st-ext.html>
- RWU “Specification Document Preparation Guidelines”

Chapter 2

Product Overview

Table 2.1: Revision History – Chapter 2

Version	Date	Author	Description
1.0	January 22, 2026	Y. Sangale	Initial Draft
1.1	April 14, 2026	Y. Sangale	Added Floating Point Unit (FPU) Specs to Key Features

2.1 Top Level Description

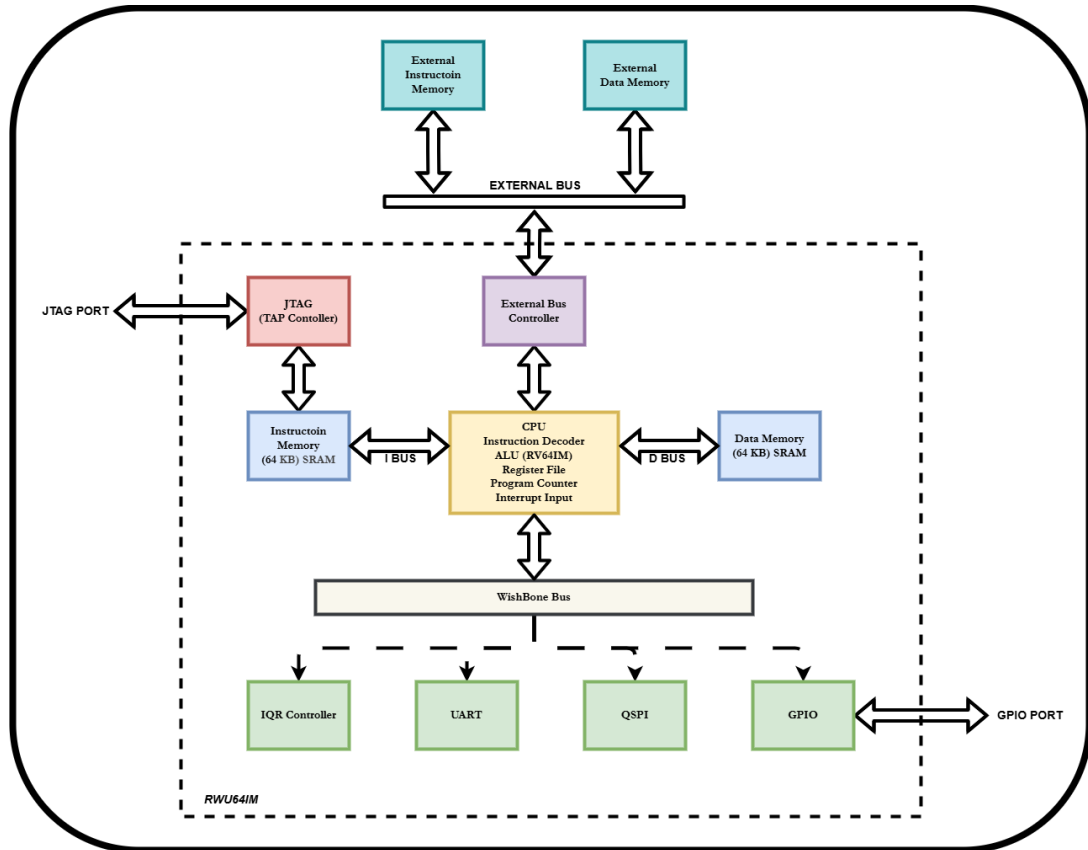


Figure 2.1: Top Level Block Diagram of RV64I System-on-Chip

2.1.1 Introduction

The designed microcontroller is a 64-bit RISC-V (RV64I) based single-core processor aimed at basic embedded applications. It operates at an internal clock frequency of 80 MHz and provides key interfaces such as General Purpose Input/Output (GPIO), Universal Asynchronous Receiver-Transmitter (UART), and Quad Serial Peripheral Interface (QSPI) for communication and JTAG for uploading Program on the controller. The architecture follows the Harvard Architecture model with separate instruction and data memories. A Wishbone-based interconnect provides standardized communication between the core and peripherals.

The peripheral subsystem includes a configurable GPIO controller with interrupt capability, a UART for serial communication, and a QSPI interface for high-speed memory or peripheral access. The system integrates a clock generation block capable of supplying phase-shifted or divided clocks to peripherals, enabling dynamic clock configuration for Pulse Width Modulation (PWM) and serial interfaces. Programming and debugging are supported through a JTAG interface. The interrupt controller supports up to eight sources with programmable trigger modes (rising, falling, both, pulse). The design serves as the foundation for future extensions including cache, DMA, and additional interfaces. The design now incorporates an IEEE 754-2008 compliant double-precision Floating-Point Unit (FPU) supporting the RV64D extension.

2.1.2 Key Features

The RISC-V System-on-Chip microcontroller includes the following key features:

- RISC-V RV64I ISA
- Addition:
 - RISC-V 64-bit Extension for Integer Multiplication and Division (RV64M)
 - RISC-V 64-bit Extension for Atomic Instructions (RV64A)
 - RISC-V 64-bit Extension for Double-Precision Floating-Point (RV64D)
- Single-core, single-pipeline processor (scalar, in-order)
- Harvard Architecture architecture with separate I-MEM and D-MEM (64 KB each)
- 80 MHz internal main clock with peripheral clock dividers
- Wishbone bus architecture for peripheral connectivity
- 32×64 -bit general-purpose registers (x0–x31), and 64-bit program counter
- 64-bit FPU, 32×64 -bit FP registers (f0–f31) registers, and internal buses.
- 8 configurable interrupt lines (Interrupt Request or Interrupt Request Line7:0 (IRQ)) with vector-based ISR dispatch
- Memory-mapped peripherals for modular extensibility
- UART, GPIO, QSPI interfaces
- JTAG for programming and on-chip debug access

2.2 Architecture Concepts: Overview

2.2.1 Functional Block Diagram

Functional blocks diagram will be included in the final version of this document.

Figure 2.1 shows the functional block diagram of the device. The architecture consists of a core, peripherals and external interface controllers. Each block communicates via a shared system Wish Bone Bus Architecture.

2.2.2 Package

Package type and pin configuration details will be provided in the final version of this document.

2.3 System View

2.3.1 Application System Description and Use Cases

The system is intended for use in low-power embedded applications such as IoT sensors, wearables, and smart home devices. A typical system setup includes power management, the microcontroller, and external communication modules such as BLE or Wi-Fi. The microcontroller is designed as a compact, easy-to-understand RV64I-based system suitable for educational purposes, Field-Programmable Gate Array (FPGA)-based prototyping, and simple embedded control applications. Its clean Harvard Architecture architecture, small peripheral set, and predictable interrupt behavior make it an ideal platform for teaching CPU pipeline concepts, memory systems, bus architectures, and peripheral interface design. Primary use cases include:

- Educational demonstrations of the RISC-V instruction set, pipeline execution, and memory-mapped I/O.
- FPGA-based prototyping of embedded systems involving UART, GPIO, or QSPI communication.
- Control-oriented applications such as sensor interfacing, simple automation systems, and custom peripherals.
- Research or coursework involving modification of the CPU pipeline, interrupt controller behavior, or clocking subsystem.

The addition of the FPU extends the range of applicable use cases to include:

- Educational demonstrations of IEEE 754 floating-point arithmetic, RISC-V F/D extension execution, and FPU pipeline behavior.
- FPGA-based prototyping of embedded systems requiring floating-point computation (e.g., sensor fusion, signal processing).

- Control-oriented applications requiring trigonometric, exponential, or high-precision arithmetic.
- Research or coursework involving FPU microarchitecture, rounding modes, and exception handling.

2.3.2 System Description

The system is organized as a Harvard Architecture-architecture microcontroller with separate instruction and data memories, where as other peripherals unified through a Wishbone interconnect for all data accesses. The CPU implements the RV64I ISA and features a straightforward single-pipeline structure. With the addition of the, RV64A, RV64M along with FPU RV64D, the CPU pipeline is extended with a dedicated floating-point execution stage that shares the register file interface but operates on the separate 64-bit floating-point register bank (f0–f31). The instruction memory (I-MEM) provides 64 KB of program space, while the data memory (D-MEM) offers an additional 64 KB region for stack, heap, and global variables.

All peripherals—including GPIO, UART, QSPI, the interrupt controller, and clock control module—are exposed via a memory-mapped address space connected through the Wishbone bus. The CPU uses standard Wishbone read/write cycles for peripheral accesses, ensuring low design complexity and full visibility for educational study.

The system operates primarily on the 80 MHz main clock (CLK_MAIN). Optional phase-shifted or divided clocks are generated through the Peripheral Clock Generator (PCG) to support PWM generation, UART baud generation, or time-sensitive external interfaces.

2.3.3 System Integration and Application Circuit

For FPGA deployments, the system integrates cleanly with minimal external circuitry. Key considerations include:

- A stable 80 MHz clock source or PLL block configured within the FPGA fabric.
- External pins for UART, GPIO, QSPI, and JTAG made available on FPGA I/O banks.
- Proper pin constraints and I/O standards set according to the FPGA development board.
- Optional external QSPI flash memory interfaced through the QSPI lines for program storage or data logging.
- Decoupling capacitors on power rails and recommended pull-ups for JTAG lines depending on board design.

A typical application circuit includes CLK_MAIN input, reset circuitry, an external QSPI flash (optional), and standard UART TX/RX connectors.

For FPGA deployments, the system integrates cleanly with minimal external circuitry. The FPU is tightly integrated into the CPU pipeline and requires no additional external components. Key considerations remain unchanged: stable 80 MHz clock, proper I/O pin constraints, and optional QSPI flash for program storage.

2.3.4 Compliances

As the design is intended for FPGA-based educational and prototype systems, no formal compliance certification is mandated. For deployment in commercial hardware, considerations should include:

- Electromagnetic compatibility (EMC) guidelines for digital systems.
- ESD protection on external GPIO, UART, and QSPI pins.
- Compliance with RoHS and REACH directives for manufacturing.
- Ensuring JTAG and debug interfaces meet IEEE 1149.1 standards.

The FPU is designed for full compliance with *IEEE 7542008* for double-precision operations. No formal certification is mandated for FPGA-based prototype use. For commercial deployment, compliance with *IEEE 7542008* must be verified through formal simulation and exhaustive corner-case testing.

The modular design and clean clocking structure facilitate compliance with common embedded hardware regulations if needed.

2.4 Architecture Concepts Overview

2.4.1 Technology

The microcontroller is intended for implementation in a modern 90 nm CMOS technology node. This process offers high logic density, low leakage, and excellent performance-per-watt characteristics, making it suitable for compact embedded microcontrollers and System-on-Chip-class designs. The design targets FPGA prototyping initially, with plans for Application-Specific Integrated Circuit (ASIC) fabrication in the future. The FPU design uses fully synchronous combinational/sequential logic compatible with automated place-and-route flows. The FPU area overhead is estimated at approximately 15-20% of the total core area, consistent with typical double-precision FPU implementations at this technology node.

Infineon's 90 nm (70 nm physical gate length) CMOS technology is designed for digital and analog circuit design. It offers advanced possibilities for low power design (triple well, low leakage transistor types). The technology has 4-9 layers of copper metallization. It offers an increase in peak performance of more than 50% and a reduction of active power consumption of more than a factor of 2. Compared to the predecessor technology twice the cell density for standard cells can be achieved in L90.

The nominal core voltage is 1.2 V, IO devices have a nominal supply voltage of 2.5 V or 1.8 V.

2.4.2 System Memory Concept

The microcontroller follows a Harvard Architecture architecture in which the Instruction Memory (I-MEM), Data Memory (D-MEM), and Register File are all directly connected to the CPU core without passing through the Wishbone bus. This separation enables parallel instruction fetch and data access, avoids structural hazards, and keeps the pipeline implementation simple and predictable.

Instruction Memory (I-MEM) is a 64KB block mapped at address range $0 \times 0000\ 0000$ – $0 \times 0000\ FFFF$. It is accessed exclusively by the instruction fetch stage of the CPU via a dedicated instruction interface. Access is read-only from the CPU perspective, allowing efficient fetch timing and minimizing critical path delays.

Data Memory (D-MEM) is a separate 64KB block mapped at address range $0 \times 0001\ 0000$ – $0 \times 0001\ FFFF$. The CPU accesses D-MEM through a dedicated data interface that supports load and store operations without involving any bus arbitration. Since D-MEM is not part of the Wishbone interconnect, data operations occur with deterministic timing independent of peripheral traffic.

The Register File consists of 32 general-purpose 64-bit registers (x0–x31) and is implemented inside the CPU core. It provides two read ports and one write port to support single-issue pipeline operation. The Register File is not memory-mapped and cannot be accessed by peripherals or external masters.

Peripheral Memory Space is accessed through the Wishbone bus and is distinct from both I-MEM and D-MEM. Only load/store instructions targeting the high address region (starting at $0 \times 1000\ 0000$) generate Wishbone transactions. This clean separation ensures that:

- program execution (I-MEM) never competes with data accesses,
- core data accesses (D-MEM) are unaffected by peripheral latency,
- Wishbone traffic originates only from memory-mapped I/O operations.

Floating-point load/store instructions (FLD, FSD) use the same D-MEM data path as integer load/store operations. The floating-point register file (f0–f31) is internal to the CPU/FPU and is not memory-mapped.

2.4.3 Software Architecture

The software architecture of the microcontroller is based on a conventional RISC-V bare-metal development flow. The hardware is implemented and verified using SystemVerilog, while Vivado is used as the primary environment for simulation, synthesis, and FPGA deployment.

Application development is performed using a RISC-V GCC toolchain configured for the RV64I ISA. Programs are written in C or assembly and compiled into machine code through the following steps:

1. C source files are compiled by the RISC-V compiler into assembly.
2. The assembler and linker generate a flat binary image targeting the I-MEM address map.

3. A custom programming tool converts the binary into a JTAG-compatible format.
4. Using the JTAG interface, the tool loads the program directly into the Instruction Memory implemented on the FPGA.

The microcontroller operates without an operating system, using a simple bare-metal startup sequence that initializes the stack pointer, configures peripherals, and optionally enables interrupts. All peripherals (GPIO, UART, QSPI, clock controller, and IRQ controller) are controlled using memory-mapped register access within the high-address I/O region. The interrupt architecture supports hardware-vector entry, simplifying handler development and reducing interrupt latency. The software architecture extends the bare-metal development flow to include floating-point support. The RISC-V GCC toolchain is configured with `-march=rv64imd` to enable double-precision FPU code generation. Floating-point programs follow the same compilation and JTAG-based loading flow as integer programs.

2.4.4 Interprocessor Communication Concept

The current microcontroller is a single-core RV64I design and therefore does not implement any interprocessor communication (IPC) mechanisms. All computation, memory access, and interrupt handling occur within the single CPU pipeline.

2.5 Functional Block Overview

The microcontroller consists of a compact set of functional modules designed around a clean Harvard Architecture architecture and a Wishbone-based peripheral subsystem. The major functional blocks include:

- **CPU Core (RV64I):** A single-issue scalar pipeline implementing the RV64I instruction set. Includes the register file, ALU, multiplication/division unit, instruction decoder, control logic, and interrupt handling interface.
- **FPU (RV64D):** IEEE 7542008 double-precision floating-point unit, tightly coupled to the CPU pipeline. Implements all required RV64D instructions including arithmetic, compare, convert, and load/store.
- **Instruction Memory (I-MEM):** A 64 KB program memory directly connected to the CPU fetch stage. Stores executable code and reset vectors.
- **Data Memory (D-MEM):** A 64 KB data RAM directly accessed by the CPU load/store unit, supporting low-latency memory operations independent of peripheral traffic.
- **Wishbone Interconnect:** A simple single-master bus used by the CPU to access all memory-mapped peripherals. Ensures modular peripheral integration and predictable transaction behavior.
- **GPIO Controller:** Provides general-purpose digital I/O pins with configurable direction and interrupt capability. Supports optional PWM via peripheral clock division.

- **UART Controller:** Implements asynchronous serial communication with programmable baud rate and interrupt-driven transmit/receive support.
- **QSPI Controller:** Provides quad-SPI communication for external flash or high-speed serial devices.
- **Interrupt Controller:** Supports eight individually configurable external interrupt lines with level, edge, and pulse triggering. Generates hardware vector addresses to accelerate ISR entry.
- **Peripheral Clock Generator (PCG):** Generates divided and phase-adjusted clock signals derived from the 80 MHz main clock for use by UART, GPIO timers, QSPI, and other peripherals.
- **JTAG Interface:** Enables programming and debugging through a standard IEEE 1149.1-compatible interface.

Each module is memory-mapped in the high-address region and integrated through the Wishbone bus, while the CPU, Register File, I-MEM, and D-MEM remain directly connected as part of the Harvard Architecture core architecture.

2.6 Pin List

External pins for the microcontroller include:

- **CLK_MAIN** – Primary 80 MHz system clock input.
- **RESET_N** – System reset, active low.
- **GPIOx** – General-purpose digital input/output pins; each configurable for direction and interrupt triggering.
- **UART_TX, UART_RX** – UART communication pins.
- **QSPI_SCLK, QSPI_CS, QSPI_IO[3:0]** – Quad-SPI interface for external flash memory.
- **IRQ7:0** – External interrupt request inputs.
- **JTAG_TCK, JTAG_TMS, JTAG_TDI, JTAG_TDO, JTAG_TRST**
 - Debug/programming pins.

Pin configuration, multiplexing options, and detailed signal descriptions will be provided in the final revision of this document.

Chapter 3

Architecture Concepts

Concepts related to power management, security, Test and debug will be fully defined in the final specification revision.

Table 3.1: Revision History – Chapter 3

Version	Date	Author	Description
1.0	January 22, 2026	Y. Sangale	Initial Draft
1.2	April 13, 2026	Y. Sangale	Added Meltdown & Spectre concept
1.1	April 21, 2026	Y. Sangale	Added FPU architecture concept

The architecture consists of a processing subsystem (CPU + FPU), peripheral subsystem, power management unit, and external interface controllers. It provides a high-level technical description of the CPU core, memory subsystem, bus structure, interrupt design, clocking strategy, FPU and register architecture. Each block communicates via a shared system bus

The logic style used here is as often as possible a "one pulse" logic. This means that all signals are valid for exactly one clock cycle, and the next state is computed based on the current state and inputs. This approach simplifies timing analysis and reduces the likelihood of glitches or metastability issues.

Figure 3.1 illustrates the pulse-based enable logic.

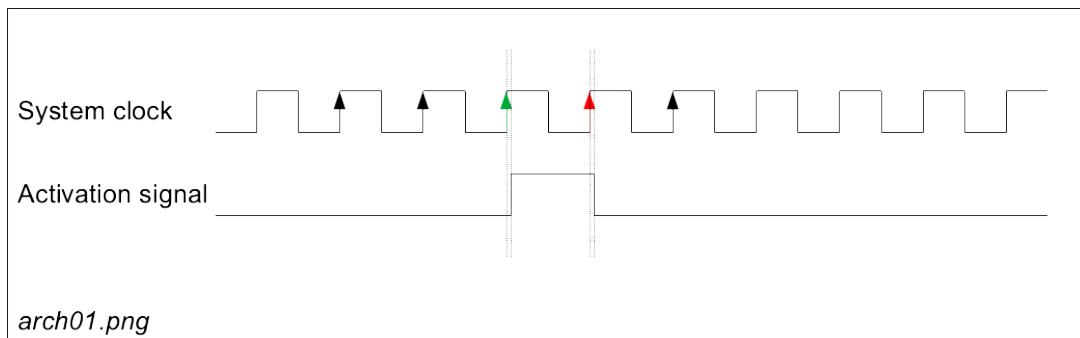


Figure 3.1: Pulse-based enable logic for synchronous control signal generation

The activation signal is generated synchronously with the system clock and is aligned to its rising edge (green). Due to synchronous logic behavior, the rising edge of the

activation signal is produced with a one-cycle latency relative to the triggering clock edge. On the subsequent rising edge of the system clock (red), the activation signal is deasserted. Similarly, the falling edge of the activation signal exhibits a small propagation delay with respect to the clock. As a result, the activation signal remains asserted for exactly one system clock period. This pulse can be used to control downstream synchronous components (e.g., a counter). The counter increments only when the activation signal is high and a rising edge of the system clock occurs (red).

3.1 Technology

The microcontroller is conceptually targeted for implementation in a modern **90 nm CMOS technology node**. This node provides a balanced compromise between integration density, power efficiency, and manufacturing cost, making it suitable for compact embedded microcontrollers and low-complexity System-on-Chip designs.

The 90 nm node enables:

- Higher operating frequencies compared to older planar CMOS nodes.
- Reduced dynamic and static power consumption.
- Dense Static Random-Access Memory (SRAM) bitcells allowing larger memories in a small silicon footprint.

Although the conceptual design and functional model are developed and validated using synthesizable Hardware Description Language (HDL) on FPGA platforms, the microarchitecture, clocking structures, and memory system were designed with 90 nm ASIC fabrication constraints in mind. This includes fully synchronous logic, controlled critical paths, and modular blocks suitable for automated place-and-route flows.

3.2 CPU Concept

Note: Unlike the original RWU-RV64I specification, which implements a single-cycle CPU, the RWU64IMD design adopts a pipelined architecture to improve performance.

The CPU core is a compact, single instruction RV64I processor, extended with a FPU implementing the RISC-V D-extension (Double-precision floating-point). The combined CPU+FPU supports:

- **Pipeline:** A simple scalar pipeline (Fetch–Decode–Execute–Writeback) without speculation or out-of-order features. [To be Implemented in the near future.](#)
- **ISA Support:** Fully compliant with RV64I + RV64M + RV64A + RV64D.
- **Integer Datapath:** 64-bit ALU, $32 \times$ 64-bit integer registers (x0–x31) and internal buses.
- **Floating-Point Datapath:** 64-bit FPU, $32 \times$ 64-bit FP registers (f0–f31) registers, and internal buses.
- **Register File:** $32 \times$ 64-bit general-purpose registers (x0–x31), with direct CPU access (no bus).

- **FP Register File:** shared with integer register file; not memory-mapped.
- **Floating-Point Status/Control Register (fcsr):** Contains rounding mode (frm) and accrued exception flags (fflags).
- **FP Memory Access:** sFLD/FSD instructions share D-MEM data path with integer load/store.
- **Memory Access Model:** Harvard Architecture architecture — separate I-MEM and D-MEM buses.
 - Dedicated instruction interface to I-MEM.
 - Dedicated data interface to D-MEM.
 - Memory-mapped I/O via Wishbone for all peripherals.
- **Program Counter:** 64-bit Program Counter (PC) with generation logic (PC +4, branch target, jump target).
- **Interrupt Integration:** Receives a single aggregated interrupt line from the Interrupt Controller and vectoring is performed as per RISC-V rules.

3.3 Memory Concept

The memory subsystem is based on a clean Harvard Architecture architecture with separated instruction and data paths.

- **Instruction Memory (I-MEM):**
 - 64 KB read-only memory for program storage.
 - Dedicated CPU instruction fetch path (non-Wishbone).
 - High-performance single-cycle access for most operations.
- **Data Memory (D-MEM):**
 - 64 KB data RAM for stack, heap, and program data.
 - Direct CPU data access (non-Wishbone).
 - Fully synchronous with CPU core.
- **Memory-Mapped I/O:**
 - Peripherals mapped in the 0x1000_0000 address region.
 - Accessed exclusively via the Wishbone peripheral bus.
- **Extensibility:**
 - Optional I-Cache and D-Cache may be added in future without changing the CPU-bus interface.

3.3.1 Memory Map

The system uses a fixed physical address map:

- **0x0000_0000 – 0x0000_FFFF:** Instruction Memory (64 KB)
- **0x0001_0000 – 0x0001_FFFF:** Data Memory (64 KB)
- **0x1000_0000 – 0x1FFF_FFFF:** Memory-Mapped Peripherals

3.4 Software Architecture Concept

The software architecture of the RWU64IMD microcontroller is based on a bare-metal execution model without an operating system. Programs are executed directly from Instruction Memory (I-MEM), with full control over hardware resources via memory-mapped registers.

- **Execution Model:** Single program execution starting from a fixed reset vector.
- **Toolchain Support:** Standard RISC-V GCC-based toolchain targeting RV64IMD ISA.
- **Memory Access:** Direct addressing of Instruction Memory, Data Memory, and peripherals.
- **Interrupt Handling:** Machine-mode interrupt handling using vector-based dispatch.

No virtual memory or operating system abstraction is provided in the baseline implementation. Future extensions may include runtime libraries or lightweight RTOS integration.

3.5 Interprocessor Communication Concept

In the baseline implementation, the RWU64IMD system is a single-core architecture and does not require interprocessor communication.

For future multi-core extensions, communication between cores is expected to be implemented using shared memory and synchronization primitives.

- **Shared Memory:** Communication via shared Data Memory or shared L2 cache.
- **Synchronization:** Atomic instructions (RV64A) used for mutual exclusion and coordination.
- **Interrupt-Based Signaling:** Inter-core interrupts may be introduced for event notification.

3.6 Interrupt Concept

The interrupt subsystem is based on a configurable multi-source interrupt controller supplying a single interrupt input to the CPU.

- **IRQ Lines:** 8 independent inputs (IRQ7:0) with per-line configuration.
- **Trigger Types:**
 - Level-high
 - Rising edge
 - Falling edge
 - Both edges
 - Pulse/one-shot
- **Edge Detection:** Implemented using per-line previous-state registers.
- **Control Registers:** Mask, type-select, and pending-status registers accessible via Wishbone.
- **Vector Computation:**

$$\text{ISR Vector} = \text{BASE_ADDR} + (\text{IRQ_ID} \times 0x10)$$

- **Priority Model:** Fixed priority (IRQ0 highest) for the initial revision.
- **CPU Interface:** Outputs a single interrupt line to the CPU's machine-mode interrupt system.

3.7 Bus Concept

The microcontroller employs a simple and predictable communication architecture centered around a **single-master bus topology**. The CPU core is the only bus master and is responsible for initiating all data and peripheral access transactions. This simplifies arbitration, reduces latency, and provides deterministic behavior suitable for education, prototyping, and low-complexity embedded applications.

The system uses two categories of interconnects:

1. **Direct CPU Connections** The following modules are connected directly to the CPU datapath and do not use the system bus:
 - **Instruction Memory (I-MEM):** Accessed through a dedicated instruction fetch interface.
 - **Data Memory (D-MEM):** Connected through a dedicated data access path for load/store instructions.
 - **Register File (RF):** Integrated directly into the CPU pipeline.

These direct connections enable single-cycle access to critical datapath components and support a clean Harvard Architecture architectural split.

2. **Wishbone-Based Peripheral Bus** All other memory-mapped peripherals are connected using a **Wishbone-compliant** slave interface. This bus is used for:

- General Purpose I/O (GPIO),
- Universal Asynchronous Receiver/Transmitter (UART),
- Quad-SPI Master (QSPI),
- Interrupt Controller,

The Wishbone bus provides standardized signaling for address, data, strobes, and acknowledgments, enabling:

- Modular and plug-and-play peripheral integration,
- Predictable, single-master timing behavior,
- Straightforward extension with new peripherals,
- Clean separation between CPU datapath and peripheral logic.

3.7.1 Wishbone Protocol Specification

The peripheral interconnect follows the Wishbone B4 classic protocol.

- **Master Signals:** ADR, DAT_W, DAT_R, WE, STB, CYC
- **Slave Signals:** ACK, STALL, ERR

A transaction is initiated by the CPU asserting **CYC** and **STB** along with a valid address. The slave responds with **ACK** to complete the transfer.

The design follows a single-master configuration, eliminating arbitration complexity and ensuring deterministic timing behavior.

3.8 System Control Concept

System control is centered around a simple and robust global reset architecture. The microcontroller implements an **external GPIO-based reset input**, which provides a direct and intuitive mechanism to reinitialize the entire system.

- **External Reset Input:** A dedicated GPIO pin is configured as a reset input. When driven **low**, it asserts a system-wide synchronous reset.
- **Reset Behavior:**
 - All CPU state (PC, registers, CSRs) is reset to defined initial vALUes.
 - Instruction Memory and Data Memory interfaces are reset.
 - All Wishbone peripherals (GPIO, UART, QSPI, IRQ controller, Clock Control, JTAG region) return to default configuration.
 - All internal state machines return to IDLE.
- **Synchronization:** The external reset signal passes through a two-stage synchronizer to ensure safe integration into the main clock domain.

- **Reset Release:** When the external GPIO reset input returns high, the system exits reset synchronously at the next active clock edge.

This minimal reset architecture provides deterministic system initialization and simplifies external control during prototyping and debugging.

3.9 Register Concept

The Register Concept defines the structure and behavior of all **memory-mapped peripheral registers** accessible through the Wishbone bus. These registers are used to configure, control, and monitor the operation of GPIO, UART, QSPI, the Interrupt Controller, the Clock Control Unit, and other on-chip peripherals.

- **Scope:** Only peripheral registers are included. CPU-internal structures such as the Register File and CSRs are part of the CPU microarchitecture and are not memory-mapped.
- **Register Types:**
 - Control registers (enable, mode select, configuration)
 - Status registers (interrupt flags, error states, peripheral activity)
 - Data registers (UART TX/RX, QSPI data buffer, GPIO input/output)
 - Configuration registers (IRQ trigger type, GPIO direction, clock divisor)
- **Access Method:** All registers are accessed via standard 32-bit or 64-bit CPU load/store instructions through the Wishbone data bus.
- **Address Alignment:** All registers are aligned to 32-bit word boundaries.
- **Reset Behavior:** On system reset, all registers revert to defined default states to ensure predictable system startup.

3.10 Clock Concept

The microcontroller uses an 80 MHz main system clock with optional peripheral clock division.

- **Core Clock:** 80 MHz clock feeding CPU, I-MEM, and D-MEM.
- **Peripheral Clocks:** Generated via a programmable divider block:
 - Divide-by-2, /4, /8, etc.
 - Per-peripheral selectable clock domains.
- **Clock Control Registers:** Accessible via Wishbone.
- **Clock Domain Crossing:** Standard two-flip-flop synchronizers used where required (e.g., UART, QSPI).

3.11 Power Concept

Section to be completed in the final document revision.

3.12 Power Concept

The RWU64IMD microcontroller is designed for low-power embedded applications.

- **Static Power:** Reduced through CMOS technology scaling.
- **Dynamic Power:** Controlled via clock gating and frequency scaling.
- **Clock Gating:** Peripheral clocks can be selectively disabled.

Future extensions may include dynamic voltage scaling and sleep modes.

3.13 System Protection and Security Concept

Section to be completed in the final document revision.

The baseline architecture does not implement hardware-enforced isolation such as privilege levels or virtual memory.

- **Execution Mode:** Single machine-mode operation.
- **Memory Protection:** No PMP or MMU in baseline design.

Future extensions may include:

- Physical Memory Protection (PMP),
- Privilege separation (User/Supervisor modes),
- Secure boot mechanisms.

3.14 Debug Concept

Section to be completed in the final document revision.

3.15 Test and Debug Concept

Debug and program loading are supported through a JTAG interface.

- **Program Loading:** Instruction Memory can be programmed via JTAG.
- **Debug Access:** External debugger can access internal registers and memory.
- **Breakpoints:** Future support for hardware breakpoints and single-step execution.

The debug interface is intended for FPGA prototyping and development and may be extended in future revisions to support advanced debugging features.

3.16 Attacks, Hazards and Mitigation Methodologies

3.16.1 Meltdown and Spectre

Transient execution attacks such as Meltdown (CVE-2017-5754) [1] and Spectre (CVE-2017-5753, CVE-2017-5715) [2] exploit the mismatch between architectural state commitment and microarchitectural execution. Instructions may execute transiently before their validity is resolved, leaving observable side-effects in shared microarchitectural components such as caches.

Three architectural preconditions must hold simultaneously for a transient execution attack to be feasible [2], [3]:

1. **Speculative or out-of-order execution:** Instructions execute before architectural checks are resolved, creating a transient execution window.
2. **Shared, timing-observable microarchitectural state:** Typically cache hierarchies that allow data-dependent timing differences to be measured.
3. **Hardware-enforced isolation boundary (attack-class specific):** Required for Meltdown and Spectre Variant 2 to cross privilege or context boundaries.

Current Implementation (Baseline Architecture)

The current RWU64IMD implementation is a strictly in-order, single-core design with no speculative or out-of-order execution. The pipeline executes one instruction at a time, and all memory accesses are resolved before subsequent dependent operations are issued.

- **No Speculation:** The absence of speculative or out-of-order execution eliminates the transient execution window required for all Spectre and Meltdown class attacks.
- **No Isolation Boundary:** The system operates entirely in machine mode without user or supervisor privilege separation, removing the architectural boundary required for Meltdown and Spectre Variant 2.
- **No Shared Cache Hierarchy:** The baseline design does not include shared caches (e.g., L2), limiting the availability of high-resolution cross-context timing side-channels.

As a result, none of the required preconditions are simultaneously satisfied, and therefore Meltdown and Spectre attacks are **not applicable** in the current implementation.

Assumed Extended Implementation (Multi-Core and Performance Enhancements)

Future extensions of the architecture may introduce:

- Multi-core execution,
- Shared L2 cache,
- Deeper pipelines with speculative execution,

- Branch prediction mechanisms (e.g., BTB, BHT),
- Optional out-of-order execution structures (e.g., ROB).

Under these assumptions, the microarchitectural landscape changes significantly:

- **Speculative Execution Introduced:** Branch prediction and deeper pipelines create transient execution windows, enabling speculative execution of instructions before branch resolution.
- **Shared L2 Cache:** A shared cache across cores introduces a measurable timing side-channel. Cache line access latency differences (hit vs. miss) can be observed using the `rdcycle` CSR, enabling data exfiltration.
- **Branch Predictor State Sharing:** Predictor structures may be shared or influenceable across execution contexts, enabling mistraining attacks.

Attack Applicability in Extended Design

- **Meltdown:** Still not applicable unless hardware-enforced privilege separation (e.g., user/supervisor modes with protected memory regions) and deferred exception handling are introduced.
- **Spectre Variant 1 (Bounds Check Bypass):** Applicable. Speculative execution combined with data-dependent memory accesses enables leakage within the same address space via cache-based side-channels.
- **Spectre Variant 2 (Branch Target Injection):** Conditionally applicable. Requires indirect branch prediction structures and cross-context predictor influence, which may arise in multi-core or multi-context environments.

Security Interpretation

The security posture of the RWU64IMD architecture is therefore highly dependent on microarchitectural features rather than the ISA itself. The baseline implementation achieves inherent resistance to transient execution attacks through architectural simplicity, while future performance-oriented extensions introduce conditions under which Spectre-class attacks become feasible.

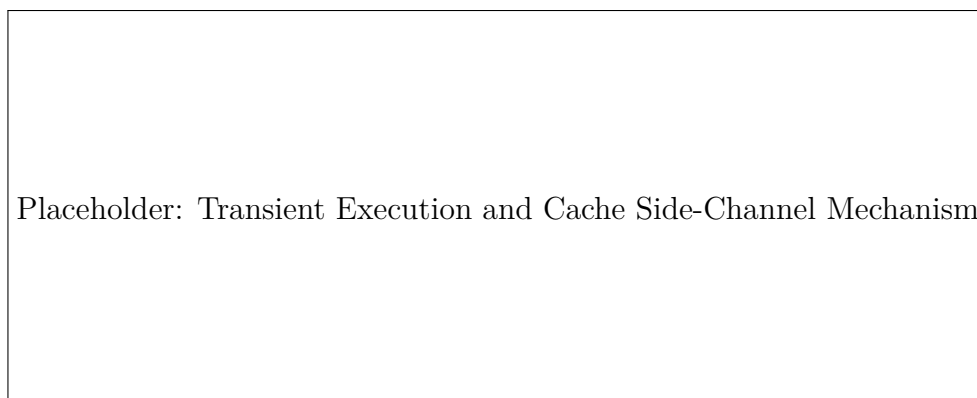


Figure 3.2: Conceptual illustration of transient execution and cache-based side-channel leakage

The detailed analysis of attack mechanisms and corresponding mitigation strategies for both baseline and extended configurations is provided in Chapter 8.

Chapter 4

Description of the Design Elements

Table 4.1: Revision History – Chapter 4

Version	Date	Author	Description
1.0	January 22, 2026	Y. Sangale	Initial Draft
1.1	April 21, 2026	Y. Sangale	Added FPU design element (Section 4.2)

This Section will be completed later.

This chapter provides a detailed technical description of all major design elements composing the RV64I 64-bit single-core microcontroller. Each sub-block is presented in terms of hardware interfaces, bus connectivity, internal registers, and its functional and structural roles within the System-on-Chip. All diagrams and tables are placeholders for later detailed schematics.

4.1 CPU Core

4.1.1 HW Interfaces to Other Blocks

The CPU Core is the central processing unit of the system, implementing the RV64IM instruction set and acting as the single bus master in the microcontroller architecture. It connects to all major subsystems as follows:

- **Instruction Memory (I-MEM):** Connected directly via the **Instruction Bus (I-Bus)** for program fetch under the Harvard Architecture architecture model.
- **Data Memory (D-MEM):** Connected directly via the **Data Bus (D-Bus)** for load/store access.
- **Wishbone Bus:** Acts as the **bus master** interfacing to all memory-mapped peripherals (GPIO, UART, QSPI, Interrupt Controller, and Clock Unit).
- **Interrupt Input:** Receives a single aggregated interrupt signal from the Vectored Interrupt Controller (VIC).
- **Clock and Reset:** Driven by the system clock from the Clock Generation Unit (CGU) and synchronized reset signal.

Figure 4.1: CPU Core interface diagram showing I-Bus, D-Bus, and Wishbone connections to system blocks.

4.1.2 Bus Interface

The CPU operates as a **Wishbone Master** for all peripheral transactions. It initiates read/write cycles with standard Wishbone control signals: `ADR_0`, `DAT_0`, `DAT_I`, `STB_0`, `ACK_I`, `CYC_0`, and `WE_0`. Instruction and data memories are accessed directly via the independent I-Bus and D-Bus, each 64-bit wide for full datapath utilization.

Figure 4.2: CPU bus architecture — direct Harvard Architecture-style connections to I-MEM/D-MEM and Wishbone master for peripherals.

4.1.3 Registers

The CPU core implements 32×64 -bit general-purpose registers (`x0-x31`) and a 64-bit program counter. Several internal control and status registers are included for interrupt and pipeline management.

Table 4.2: CPU Core Register Summary

Register	Description
PC	64-bit Program Counter for sequential and branch control
IR	Instruction Register holding current fetched instruction
SR	Status Register with control and flag bits
IBASE	Base address for interrupt vector table
MSTATUS	Machine Status Register (RV64I subset)

4.1.4 Functional Overview

The CPU executes instructions using a deterministic scalar pipeline that supports the RISC-V RV64IM base instruction set. Internally, it consists of the following major sub-modules:

- **Instruction Decoder:** Decodes opcodes fetched from I-MEM and generates control signals for ALU and memory.
- **Program Counter (PC):** Maintains the address of the current instruction and computes sequential or branch targets.
- **ALU:** Performs arithmetic, logical, shift, and comparison operations; supports integer multiply/divide.
- **Register File:** Provides two read ports and one write port for operand access and result storage.
- **Interrupt Input:** Interfaces with VIC to handle asynchronous interrupt requests.

Each instruction is executed in a simple in-order flow without speculation or pipeline hazards beyond single-cycle stalls.

Figure 4.3: CPU Core internal structure showing Instruction Decoder, ALU, Program Counter, and Register File.

4.1.5 Structural Overview

The CPU architecture follows a Harvard Architecture model with:

- Dedicated instruction path (I-Bus) for fetching 64-bit instructions.
- Dedicated data path (D-Bus) for memory and peripheral data transactions.
- Unified Wishbone interface for peripheral control and configuration.

The internal control path coordinates fetch, decode, execute, and memory operations. Instruction flow synchronization ensures deterministic timing suitable for embedded applications.

Figure 4.4: Simplified Harvard Architecture architecture connection of CPU to I-MEM, D-MEM, and Wishbone peripherals.

4.1.6 Interrupts

A single interrupt input line connects to the VIC. The CPU jumps to a vector address computed as:

$$\text{ISR Vector} = \text{IBASE} + (\text{IRQ_ID} \times 0x10)$$

Upon completion, control returns to the instruction following the interrupted instruction.

4.1.7 Clock and Reset

The CPU operates at an 80 MHz clock supplied by the CGU. Reset initializes the PC to 0x0000_0000, clears all pipeline stages, and resets the internal status registers.

4.1.8 Voltage Class

Core operates in the 3.3 V logic domain.

4.2 Floating-Point Unit (FPU)

4.2.1 Introduction

This section specifies the Floating-Point Unit (FPU) added to the RV64I_RWU microcontroller as an integral extension. The FPU implements the RISC-V D-extension (RV64D) and is fully compliant with the IEEE 754-2008 Standard for Floating-Point Arithmetic in double-precision (64-bit) format. The FPU is tightly coupled to the CPU

pipeline, sharing the instruction fetch and decode stages, and operates on a dedicated 64-bit floating-point register file (f0–f31) separate from the integer register file.

The FPU is designed for deterministic, in-order execution consistent with the single-pipeline philosophy of the microcontroller. All floating-point operations are performed in hardware with no software emulation fallback. The FPU supports all mandatory IEEE 754-2008 operations, all five rounding modes, and all five floating-point exception flags.

4.2.2 Compliance and Standards

The FPU is designed to comply with the following standards and specifications:

Standard / Specification	Version / Reference	Compliance Level
IEEE 754-2008	Standard for Floating-Point Arithmetic	Full (double-precision)
RISC-V Unprivileged ISA	Volume I, D-Extension (RV64D)	Full
RISC-V Unprivileged ISA	Volume I, F-Extension (RV64F)	Full (subset of D)
RISC-V Privileged ISA	fcsr register (frm + fflags)	Full

4.2.3 Floating-Point Data Formats

Double-Precision Format (IEEE 754-2008 binary64)

The primary data format supported by the FPU is the 64-bit double-precision binary floating-point format as defined in IEEE 754-2008:

Field	Bits	Description
Sign (S)	1	0 = positive, 1 = negative
Exponent (E)	11	Biased exponent, bias = 1023 (0x3FF)
Mantissa / Fraction (M)	52	Fractional part of significand
Total	64	64-bit double-precision floating-point word

Single-Precision Format (IEEE 754-2008 binary32)

The FPU also supports single-precision (32-bit) operations as required by the RISC-V F-extension (RV64F).

Field	Bits	Description
Sign (S)	1	0 = positive, 1 = negative
Exponent (E)	8	Biased exponent, bias = 127 (0x7F)
Mantissa / Fraction (M)	23	Fractional part
Total	32	NaN-boxed to 64-bit in FP registers

Special Values

Value	Exponent	Mantissa	Description
$\pm$ Zero	0x000	0	Positive and negative zero
Subnormal	0x000	Non-zero	Gradual underflow
$\pm$ Infinity	0x7FF	0	Overflow / divide-by-zero
qNaN	0x7FF	MSB=1	Quiet NaN
sNaN	0x7FF	MSB=0	Signalling NaN
Canonical NaN	0x7FF	0x8000000000000000	0x7FF8000000000000

4.2.4 Floating-Point Register File

Register Organization

The FPU provides 32 dedicated 64-bit floating-point registers (f0–f31).

Register	ABI Name	Description
f0–f7	ft0–ft7	Temporaries
f8–f9	fs0–fs1	Saved
f10–f11	fa0–fa1	Arguments/return
f12–f17	fa2–fa7	Arguments
f18–f27	fs2–fs11	Saved
f28–f31	ft8–ft11	Temporaries

Floating-Point Control and Status Register (fcsr)

Field	Bits	CSR	Description
NX	0	0x001	Inexact
UF	1	0x001	Underflow
OF	2	0x001	Overflow
DZ	3	0x001	Divide by Zero
NV	4	0x001	Invalid Operation
frm	[7:5]	0x002	Rounding Mode

4.2.5 Rounding Modes

Encoding	Mnemonic	IEEE Mode	Description
000	RNE	roundTiesToEven	Default
001	RTZ	roundTowardZero	Truncate
010	RDN	roundTowardNegative	Floor
011	RUP	roundTowardPositive	Ceiling
100	RMM	roundTiesToAway	Away from zero
111	DYN	–	Dynamic

4.2.6 Exception Handling

IEEE 754 Exception Flags

Exception	Bit	Condition
Invalid (NV)	4	NaN, invalid ops
Divide by Zero (DZ)	3	Non-zero / 0
Overflow (OF)	2	Exceeds range
Underflow (UF)	1	Subnormal
Inexact (NX)	0	Rounded

Default Results

- Invalid: Canonical NaN
- Divide by Zero: $\pm\infty$
- Overflow: $\pm\infty$ or max value
- Underflow: Subnormal or zero
- Inexact: Rounded result

Exception Flag Encoding Convention

All five exception flags are packed into a single 5-bit vector by every arithmetic unit, using a fixed bit order consistent with the `fflags` CSR layout:

$$\text{fflags} = \{ \text{NV}[4], \text{DZ}[3], \text{OF}[2], \text{UF}[1], \text{NX}[0] \}$$

Each functional unit drives this vector combinationally as part of its result. On instruction commit, the active unit's flag vector is OR-ed (sticky-accumulated, per IEEE 754 §7) into the `fcsr.fflags` field – flags persist across instructions until explicitly cleared by software.

4.2.7 Supported Instructions

Arithmetic Instructions

FADD.D, FSUB.D, FMUL.D, FDIV.D, FSQRT.D, FADD.S, FSUB.S, FMUL.S, FDIV.S, FSQRT.S

*Note: Fused multiply-add instructions (**FMADD.D**, **FMSUB.D**, **FNMADD.D**, **FNMSUB.D**) are not yet implemented in the current RTL. No dedicated fused-multiply-add module exists, and `fpu_ctrl1.sv` does not decode these opcodes. They are listed under Future Extensions (Section 4.2.14).*

Load/Store

FLD, FSD, FLW, FSW

Note: these instructions bypass the FPU arithmetic datapath entirely. They are decoded but routed through the standard D-MEM load/store path shared with integer loads/stores; only the destination register file (integer vs. floating-point) differs.

Comparison

FEQ.D, FLT.D, FLE.D, FEQ.S, FLT.S, FLE.S

Min / Max

FMIN.D, FMAX.D, FMIN.S, FMAX.S

Note: implemented as a dedicated unit separate from the arithmetic core (see Section 4.2.8.2), not as part of the add/subtract datapath.

Conversion (Integer $\leftrightarrow$ Floating-Point)

FCVT.W.D, FCVT.WU.D, FCVT.L.D, FCVT.LU.D, FCVT.D.W, FCVT.D.WU, FCVT.D.L, FCVT.D.LU, and the corresponding single-precision FCVT.*.S / FCVT.S.* variants

Conversion (Floating-Point $\leftrightarrow$ Floating-Point)

FCVT.S.D, FCVT.D.S

Note: this category was missing from the previous revision. It performs precision conversion between single and double formats, including NaN-boxing on widen and Guard/Round/Sticky-based rounding on narrow.

Move / Sign-Injection

FMV.X.D, FMV.D.X, FMV.X.W, FMV.W.X, FSGNJ.D, FSGNJN.D, FSGNJX.D, FSGNJ.S, FSGNJN.S, FSGNJX.S

Note: the single-precision move variants (FMV.X.W, FMV.W.X) and single-precision sign-injection variants were implemented but omitted from the previous revision's list.

Classify

FCLASS.D, FCLASS.S

4.2.8 FPU Microarchitecture**Upcast-Compute-Downcast Methodology**

Rather than implementing two independent arithmetic datapaths – one native to single precision and one native to double precision – every arithmetic unit (`fpu_addsub`, `fpu_mul`, `fpu_div`, `fpu_sqrt`) implements only the double-precision datapath. A single-precision operand is upcast to double precision before entering this shared pipeline; the double-precision result is downcast back to single precision only when the instruction's format field requests it:

S operand $\xrightarrow{s_to_d()}$ D operand $\xrightarrow{\text{shared double-precision pipeline}}$ D result $\xrightarrow{d_to_s_nanbox()}$ S result (if format = S)

This is IEEE 754-correct, not merely an implementation shortcut: the standard defines `FADD.S` as “compute the exact mathematical result, then round once to single precision.” Double precision carries 53 significand bits versus single's 24 – sufficient headroom

to represent the exact intermediate result before any rounding occurs. Rounding once, at the final downcast step, therefore yields the identical, IEEE-correct result that a native single-precision datapath would produce, with no double-rounding error.

The `s_to_d/d_to_s_nanbox` conversion functions are implemented locally within each of the four arithmetic modules rather than factored into a shared module – a deliberate tradeoff of minor code duplication in exchange for fully self-contained, independently verifiable units.

Pipeline Integration

Fetch → Decode → FP Execute → Writeback

Execution Latencies

Correction from previous revision: the FPU does not currently implement multi-cycle execution for any operation. All eleven functional units (`fpu_addsub`, `fpu_mul`, `fpu_div`, `fpu_sqrt`, `fpu_cmp`, `fpu_cvt`, `fpu_cvt_fmt`, `fpu_sign`, `fpu_minmax`, `fpu_mv`, `fpu_classify`) are purely combinational and execute within a single clock cycle, alongside the integer ALU, inside the EXEC state of the `top_npl` Finite State Machine (FSM).

Table 4.3: FPU Unit Latency (Current Implementation)

Unit	Latency	Note
<code>fpu_addsub</code> , <code>fpu_mul</code> , <code>fpu_cmp</code> , <code>fpu_cvt</code> , <code>fpu_cvt_fmt</code> , <code>fpu_sign</code> , <code>fpu_minmax</code> , <code>fpu_mv</code> , <code>fpu_classify</code>	0 cycles	Fully combinational
<code>fpu_div</code>	0 cycles	Combinational; exposes <code>clk/valid_in/valid_out</code>
<code>fpu_sqrt</code>	0 cycles	Non-restoring bit-by-bit algorithm, unrolled

Synthesis caveat: the combinational divider and square-root units produce a long critical path and are expected to require timing closure work (or conversion to a multi-cycle/pipelined implementation using the existing handshake ports) for any target above simulation. This is a documented, known limitation of the current implementation, not an oversight.

A future pipelined revision targeting non-zero latencies (e.g. FADD: 3, FMUL: 4, FDIV: 20–24, FSQRT: 25–30 cycles) is tracked under Future Extensions (Section 4.2.14) and is not part of the current implementation.

Functional Unit Map

The FPU is composed of eleven independently instantiated submodules, dispatched from a single combinational control block (`fpu_ctrl`). All units execute in parallel every cycle; a result multiplexer, driven by the decoded opcode, selects the active unit's output.

Table 4.4: FPU Functional Unit Map

Module	Instructions Handled	Output Destination
fpu_addsub	FADD, FSUB	FP regfile
fpu_mul	FMUL	FP regfile
fpu_div	FDIV	FP regfile
fpu_sqrt	FSQRT	FP regfile
fpu_cmp	FEQ, FLT, FLE	Integer regfile
fpu_cvt	FCVT.* (int $\leftrightarrow$ float)	Integer or FP regfile
fpu_cvt_fmt	FCVT.S.D, FCVT.D.S	FP regfile
fpu_sign	FSGNJ, FSGNJN, FSGNJX	FP regfile
fpu_minmax	FMIN, FMAX	FP regfile
fpu_mv	FMV.X.D, FMV.D.X, FMV.X.W, FMV.W.X	Integer or FP regfile
fpu_classify	FCLASS	Integer regfile

The destination register file is selected by the `to_int` signal, described in Section 4.2.9.

FP Register File

32 $\times$ 64-bit, 2 read ports, 1 write port

Operand Forwarding

Supported for single-cycle ops

NaN Handling

IEEE 754 compliant

Subnormal Handling

Fully supported (no flush-to-zero)

4.2.9 Hardware Interfaces

Correction from previous revision: the signal names listed previously (`fpu_op`, `fpu_rs1`, `fpu_rs2`, `fpu_ready`) do not correspond to the actual `fpu_top` port list. The real interface is:

Table 4.5: fpu_top Port List

Signal	Description
inst	Decoded instruction enum, shared with the integer decode stage
fp_funct5	Sub-operation select field (extracted from the opcode)
fp_rm	Rounding mode (resolved from <code>fcsr.frm</code> if dynamic)
fp_cvt_toint	Selects integer-destination conversion direction
fp_cvt_word	Selects 32-bit vs. 64-bit integer conversion width
fp_cvt_signed	Selects signed vs. unsigned integer conversion
fp_fmt	Selects single vs. double precision format
fp_sgn_op	Selects FSGNJ / FSGNJN / FSGNJX variant
fp_min_sel	Selects FMIN vs. FMAX
operand_a, operand_b	64-bit floating-point operands
int_operand	Integer operand (for int→float conversion)
result	64-bit result (floating-point or integer, per <code>to_int</code>)
to_int	Selects integer regfile (1) vs. FP regfile (0) as write-back destination
fflags	5-bit exception flag vector, see Section 4.2.6
div_valid_in, div_ready, div_valid_out	Handshake stub ports for the divider's future pipelined revision; currently always-ready/combinational (see Section 4.2.8)

4.2.10 Floating-Point CSR Registers

Addr	Name	Width	Access	Reset	Description
0x001	fflags	32	RW	0	Exception flags
0x002	frm	32	RW	0	Rounding mode
0x003	fcsr	32	RW	0	Combined

4.2.11 Address Map and Integration

FPU is internal, not memory-mapped.

4.2.12 Power Considerations

Always powered; clock gating planned.

4.2.13 Integration with top_npl

The FPU is integrated as a second execution unit running in parallel with the integer ALU, sharing the same decode stage and committing in the same cycle.

- **Shared decode:** the decoded instruction feeds both `alu_top` and `fpu_top` in the same cycle; no separate FP decode stage exists.
- **Parallel register file reads:** a dedicated 32×64-bit FP register file is read every cycle alongside the integer register file, both asynchronously, so floating-point operands are available with no additional read latency.

- **Single-cycle execution:** `fpu_top` evaluates combinationally within the **EXEC** state of the `top_npl` FSM – the same cycle used by the integer ALU.
- **Write-back routing:** the `to_int` signal selects the destination register file via two mutually exclusive write-enable equations:

```
int_we = commit && !trap && reg_write &&  
        (!is_fp || (is_fp && to_int));
```

```
fp_we  = commit && !trap && reg_write &&  
        (fp_load || (is_fp && !fp_load && !to_int));
```

- **Load/store bypass:** FLD/FSD are decoded to `FPU_SEL_PASS` in `fpu_ctrl` and do not enter the FPU arithmetic datapath; they reuse the same D-MEM load/store path as integer loads/stores, differing only in destination register file.
- **Exception flag accumulation:** on every committed floating-point instruction, the active unit's `fflags` output is OR-ed into `fcsr.fflags` (sticky accumulation, per Section 4.2.6).

4.2.14 Verification and Test Strategy

Unit-Level Simulation

SystemVerilog testbenches

IEEE Compliance Testing

Berkeley TestFloat

Integration Testing

RV64I toolchain tests

Regression Testing

CTest framework

4.2.15 Future Extensions

- Fused multiply-add instructions (FMADD.D, FMSUB.D, FNMADD.D, FNMSUB.D)
– not yet implemented
- Pipelined/multi-cycle FDIV and FSQRT using the existing handshake ports, targeting the latencies listed in Section 4.2.8
- Flush-to-Zero
- Quad precision (Q-extension)
- Clock gating

- Hardware exception trapping
- Transcendental functions

Chapter 5

Validation and Test

Table 5.1: Revision History – Chapter 6

Version	Date	Author	Description
1.0	January 22, 2026	Y. Sangale	Initial Draft

This Section will be completed later.

List the memory map and all registers with its addresses and reset values in tabluar format. This includes all peripheral registers and CPU control and status registers (CSRs).

Chapter 6

Toolchain

Table 6.1: History / Revision – Chapter 5

Version	Date	Author	Remarks
1.0	January 22, 2026	Y. Sangale	Initial template draft

6.1 Introduction of the Toolchain

The development of a modern System-on-Chip (SoC) requires a robust, automated, and developer-friendly toolchain that seamlessly integrates hardware design, firmware development, simulation, testing, and documentation. For the **RV64I_RWU** project, a comprehensive toolchain was designed and extended to support the full development lifecycle of a 64-bit RISC-V processor core.

The toolchain is centered around a **modular CMake-based build system** and is complemented by a custom-developed **Visual Studio Code (VS Code) extension**. This combination enables both command-line-driven workflows for automation and graphical workflows for developer productivity. The primary goals of the toolchain are:

- Reproducible and cross-platform builds
- Automated firmware compilation and hardware simulation
- Scalable regression testing using CTest
- Reduced onboarding effort for new developers
- Tight integration between software, hardware, and verification flows

By abstracting complex build and simulation commands behind well-defined interfaces, the toolchain significantly improves usability while maintaining full transparency and flexibility for advanced users.

6.2 Technologies Used

The toolchain is built using industry-standard tools and technologies to ensure portability, maintainability, and long-term scalability.

6.2.1 Build and Automation Technologies

- **CMake (≥ 3.20):** Used as the primary build system generator. A modular CMake architecture enables auto-discovery of RTL, firmware, and test files while maintaining a clean separation of concerns.
- **CTest:** Integrated with CMake to provide automated unit, integration, firmware, and IP-level regression testing with support for parallel execution.
- **RISC-V GNU Toolchain:** The `riscv64-unknown-elf-gcc` compiler is used for compiling firmware written in assembly and C for the RV64I architecture.

6.2.2 Simulation and Verification Tools

- **Xilinx Vivado XSIM:** Used for RTL simulation, supporting both console-based execution for automation and GUI-based execution with waveform visualization.
- **TCL Scripts:** Employed to standardize waveform setup, simulation control, and batch execution in XSIM.

6.2.3 Development and Integration Tools

- **Visual Studio Code:** Serves as the primary development environment.
- **Node.js:** Used to develop a custom VS Code extension that provides an integrated user interface for the toolchain.
- **Git:** Used for version control and collaborative development.
- **Sphinx:** Used for generating HTML and PDF documentation directly from the project source.

6.3 Usage of the Toolchain

The RV64I toolchain supports two primary usage modes: a terminal-based workflow and an IDE-based workflow using the VS Code extension.

6.3.1 Terminal-Based Usage

The terminal workflow provides full control over the build and simulation process and is well suited for automation, scripting, and continuous integration environments.

Configuration and Build

```
cmake -S . -B build -G "MinGW Makefiles"
cmake --build build
```

This step configures the project, detects available tools, and builds all auto-discovered targets.

Firmware Compilation

```
cmake --build build --target fw_asm_instr06addi
```

Firmware targets are generated automatically based on the source files present in the firmware directory.

Simulation Execution

Console Mode:

```
xsim instr06addi_snapshot -runall
```

GUI Mode with Waveforms:

```
xsim instr06addi_snapshot --gui --tclbatch sim/config/xsim_cfg2.tcl
```

Regression Testing

```
ctest --test-dir build -j
```

CTest enables parallel execution of unit, integration, firmware, and IP-level tests.

6.3.2 VS Code Extension Usage

To improve usability and reduce the cognitive load of complex command sequences, a custom VS Code extension named **rwu-rv64i-toolchain** was developed.

Setup Wizard

The extension provides an interactive setup wizard that:

- Detects installed tools (CMake, RISC-V GCC, XSIM, Git)
- Reports missing components
- Allows manual configuration of executable paths

This significantly simplifies first-time setup, especially on Windows systems.

Build and Simulation Commands

The extension exposes high-level commands via the Command Palette, including:

- Configure and Build All
- Build Specific Target (compile only)
- Run Simulation (console mode)
- Run Simulation (GUI with waveforms)

Target names are automatically inferred from the currently opened file, reducing manual input errors.

Regression Testing Support

The extension integrates CTest commands, enabling:

- Execution of specific tests
- Full library regression testing
- Parallel test execution

All test results are displayed directly in the VS Code terminal for immediate feedback.

6.4 Further Development of the Toolchain

While the current toolchain is fully functional, several extensions and improvements are planned.

6.4.1 CMake Enhancements

Future improvements to the CMake-based build system include:

- Support for multiple simulators beyond XSIM
- Improved dependency visualization and reporting
- Enhanced cross-compilation support for additional RISC-V variants
- Integration with continuous integration (CI) pipelines

6.4.2 VS Code Extension Enhancements

Planned enhancements to the VS Code extension include:

- Graphical dashboards for build and test status
- Integrated waveform viewers
- Per-target configuration profiles
- Deeper integration with documentation and architecture diagrams

These improvements aim to further reduce development time and improve the overall developer experience.

Chapter 7

Test And Debug

Table 7.1: Revision History – Chapter 5

Version	Date	Author	Description
1.0	January 22, 2026	Y. Sangale	Initial Draft

This Section will be completed later.

Describe how the SoC will be tested or simulated for functionality. Include details on testbenches, simulation tools, and debugging interfaces.

Chapter 8

Attacks, Hazards and Mitigation Methodologies

Table 8.1: Revision History – Chapter 8: Attacks, Hazards and Mitigation Methodologies

Version	Date	Author	Description
1.0	April 13, 2023	Y. Sangale	Initial Draft

This chapter analyses the security properties of the RWU64IMD microcontroller with respect to transient execution attacks, documents the conditions under which the architecture is and is not vulnerable, and specifies both the functional verification strategy and the security test approach.

8.1 Meltdown and Spectre

8.1.1 Background and Threat Model

Meltdown and Spectre are classes of *transient execution attacks*: they exploit instructions that are executed microarchitecturally but never committed to the architectural state [1], [2]. The resulting microarchitectural state changes — specifically modifications to the cache hierarchy — persist after the transient instructions are discarded, providing a covert channel from which secret data can be recovered by timing measurement.

The threat model assumes an attacker capable of executing unprivileged code on the system, controlling the inputs to shared code paths, and measuring execution time via the `rdcycle` counter.

8.1.2 Architectural Preconditions

Attack feasibility is determined by the simultaneous presence of the following conditions [2], [3], [4]:

1. **Speculative / out-of-order (OoO) execution.** A Reorder Buffer (ROB) allows the pipeline to execute instructions ahead of their architectural validation. Without this, no transient execution window exists.

2. **Hardware branch predictor.** Required for Spectre: the predictor enables speculative execution along a predicted but architecturally incorrect path. The BTB (Branch Target Buffer) is additionally required for Variant 2.
3. **Shared, timing-observable cache.** Both Meltdown and Spectre encode transiently accessed data into cache state. The encoding is exfiltrated by measuring the access latency difference between a cache hit (≤ 15 cycles at 80 MHz) and a cache miss to DRAM (≥ 150 cycles) [2]. A shared L2 cache, accessible across all cores, provides a persistent and observable side-channel.
4. **Hardware-enforced isolation boundary (attack-specific).** Meltdown additionally requires a privilege boundary and a deferred page-fault mechanism. Spectre Variant 2 additionally requires separate execution contexts sharing predictor state.

Removing any single required precondition is sufficient to invalidate the corresponding attack class.

8.1.3 Meltdown (CVE-2017-5754)

Mechanism

Meltdown exploits the temporal gap between the speculative execution of a faulting memory load and the enforcement of the access-control decision that generates the fault [1]. The pipeline issues the load and continues executing dependent instructions before the MMU has resolved the permission check. During this transient window the loaded value — drawn from a protected memory region — propagates into a cache-encoding instruction:

```

1 /* Step 1: faulting load from privileged address */
2 unsigned char secret = *(unsigned char *)kernel_addr;
3
4 /* Step 2: encode secret into cache state (transient) */
5 temp = probe_array[secret * 512];

```

Listing 8.1: Meltdown-style transient execution example

The subsequent page fault flushes the ROB and restores the architectural register state, but the cache-line fill caused by `probe_array` access is not reverted. The attacker recovers the secret by measuring access latency across all 256 probe-array indices (Flush+Reload technique [1], [5]):

```

1 uint64_t t0, t1;
2 for (int i = 0; i < 256; i++) {
3     asm volatile ("rdcycle_%" : "=r"(t0));
4     volatile uint8_t v = probe_array[i * 512];
5     asm volatile ("rdcycle_%" : "=r"(t1));
6     if ((t1 - t0) < HIT_THRESHOLD) recovered_byte = i;
7 }

```

Listing 8.2: Meltdown secret recovery example

A placeholder for the Meltdown attack flow diagram is reserved below:

[Placeholder: Meltdown attack flow — Flush → Faulting Load → Speculative Encode → ROB Flush → Reload → Secret Recovery]

Figure 8.1: Meltdown attack execution flow on an OoO processor with virtual memory.

Applicability to the RWU64IM Design

The RWU64IM implements physical addressing exclusively. There are no page tables, no MMU, no `satp` register, and no page-fault exceptions. Every load instruction targets a physical address and succeeds architecturally; no faulting access is generated and consequently no transient fault window is created.

In a multi-core extension without virtual memory the situation is unchanged: adding cores and a shared L2 cache does not introduce the page-fault mechanism that Meltdown requires. The shared L2 does increase the persistence and cross-core observability of the cache side-channel, but this is a necessary condition for Meltdown only after the other preconditions are satisfied.

Table 8.2: Meltdown Precondition Mapping for RWU64IM (Extended Configuration)

Required Precondition	Status in RWU64IM	Impact
OoO execution / ROB	Present (per core)	Precondition met
Virtual memory / MMU	Absent	Attack impossible
Privilege separation (U/S)	Absent	Attack impossible
Page-fault mechanism	Absent	Attack impossible
Shared cache (L1/L2)	Present	Channel present

Result: Meltdown is structurally not applicable to the RWU64IM in any single-core or multi-core configuration that retains physical addressing and a single machine-mode privilege level.

8.1.4 Spectre (CVE-2017-5753 / CVE-2017-5715)

Mechanism — Variant 1 (Bounds Check Bypass)

Spectre Variant 1 exploits the branch predictor’s training history to induce speculative execution of an out-of-bounds memory access [2]. The attacker repeatedly invokes a victim function with in-bounds inputs to train the predictor, then supplies an out-of-bounds index. The predictor speculatively enters the guarded block, executing:

```

1  /* Victim code (shared function) */
2  if (x < array1_size) {
3      uint8_t val = array1[x];          /* out-of-bounds if x is
      malicious */

```

```

4   temp = probe_array[val * 512];    /* encodes val into cache
    */
5 }

```

Listing 8.3: Spectre Variant 1 victim code

The branch subsequently resolves as not-taken; the ROB discards the speculative results. The cache-line fill caused by `probe_array[val*512]` persists. The secret byte is recovered by the same Flush+Reload timing measurement as Meltdown.

Mechanism — Variant 2 (Branch Target Injection)

Variant 2 poisons the Branch Target Buffer to redirect an indirect branch in a victim execution context to an attacker-chosen code gadget [2]. The gadget accesses sensitive data, encodes it into cache state, and the attacker recovers it via timing. This attack requires separate, isolated execution contexts sharing a physical branch predictor.

A placeholder for the Spectre attack flow diagram is reserved below:

[Placeholder: Spectre Variant 1 attack flow — Train Predictor → Mispredict → Speculative OoB Access → Cache Encode → Rollback → Timing Measurement]

Figure 8.2: Spectre Variant 1 execution flow: bounds-check bypass via branch misprediction.

Applicability to the RWU64IM Design

Table 8.3 maps each Spectre precondition to the extended RWU64IM configuration.

Table 8.3: Spectre Precondition Mapping for RWU64IM (Extended Multi-Core Configuration)

Precondition	V1	V2	Status in RWU64IM
Branch predictor (per-core)	Required	Required	Present
OoO execution / ROB (per-core)	Required	Required	Present
Shared cache (L2 across cores)	Required	Required	Present
Isolation boundary (HW-enforced)	Optional ^a	Required	Absent
Separate execution contexts	No	Required	Absent
Shared BTB across contexts	No	Required	Absent ^b

^a Required for a meaningful attack; see Variant 1 qualification below.

^b All cores share machine-mode; no separate victim context exists.

Variant 1 — single trust domain. All software on the RWU64IM executes in machine mode with uniform physical memory access. In this configuration there is no security boundary to cross; leakage across a non-existent boundary has no security consequence.

Variant 1 — multi-component software (conditional risk). If the system runs co-resident software components (e.g. a cooperative RTOS with multiple tasks sharing the physical address space), a software-defined trust boundary emerges. In this scenario Variant 1 is feasible: the hardware branch predictor can be trained by one task to induce speculative out-of-bounds reads in another task's execution context. The shared L2 cache amplifies the side-channel because a cache-line fill in one core's execution remains observable to all other cores via timing. **This is the primary residual risk of the multi-core extension.**

Variant 2. Variant 2 requires a distinct, isolated victim execution context and shared predictor state across context boundaries (e.g. user/kernel or two OS processes). No such contexts exist in the RWU64IM privilege model; all code shares machine mode. Variant 2 is not applicable.

8.1.5 Cache Timing Side-Channel

A cache timing side-channel is present independent of Meltdown/Spectre applicability [4], [5]. The conditions are:

- Per-core L1 data cache with data-dependent fill behaviour.
- Shared L2 cache accessible by all cores, whose eviction pattern reflects cross-core memory access activity.
- `rdcycle` CSR accessible in machine mode, providing cycle-accurate timing resolution sufficient to distinguish a cache hit from a miss.

The cross-core observability introduced by the shared L2 extends the attack surface relative to the single-core baseline: a cache-line filled by speculative execution on core A is observable from core B through timing, enabling cross-core Flush+Reload or Prime+Probe measurements [5].

8.1.6 Countermeasures

Software Countermeasures

Speculation barriers (fence). Insert a `fence` instruction immediately after bounds-check branches at software trust boundaries. The fence prevents the pipeline from issuing instructions beyond that point until all prior instructions have been architecturally committed, collapsing the speculative execution window:

```
1 if (x < size) {  
2     asm volatile ("fence");  
3     val = array[x];  
4 }
```

Listing 8.4: Speculation barrier example

Index masking (branch elimination). Replace conditional bounds checks with bitwise masking for power-of-two array sizes. This eliminates the conditional branch and removes the branch predictor dependency entirely:

```

1 x    = x & (size - 1);    /* no branch: no misprediction possible
   */
2 val = array[x];

```

Listing 8.5: Index masking example

Constant-time access patterns. Security-sensitive routines (cryptographic key operations, inter-task shared buffers) must access data unconditionally and select results using arithmetic masks rather than data-dependent branches, eliminating data-dependent cache footprints.

Cycle-counter restriction. Setting `mcounteren.CY = 0` prevents machine-mode code from delegating `rdcycle` access to less-privileged contexts (relevant if U-mode is added in a future revision). On the current single-privilege design this reduces the precision available to tasks within a cooperative multitasking environment if the scheduler explicitly gates the counter.

Hardware Countermeasures

L2 cache partitioning. The shared L2 cache should be partitioned by software component (e.g. by task or core) using cache-way allocation, if supported by the cache controller implementation. This prevents cross-component or cross-core eviction-based observation. Each partition must be sized to accommodate the working set of the assigned component.

Speculative cache-fill suppression. The strongest hardware mitigation is to suppress cache-line fills from load instructions that are subsequently flushed from the ROB without architectural commitment. This aligns microarchitectural cache state with the architectural commit boundary, eliminating the persistent side-channel that all transient execution attacks depend on. Implementation requires tagging each speculative load in the ROB with a *speculative* flag and gating the cache-fill write until the instruction reaches the commit stage:

Applicability Summary

Table 8.4: Countermeasure Applicability — RWU64IM Extended Configuration

Countermeasure	Targets	Type	Cost	Effectiveness
fence barriers	Spectre V1	SW	Perf. stall	High
Index masking	Spectre V1	SW	Negligible	High
Constant-time access	Cache side-channel	SW	Moderate	High
<code>mcounteren</code> restriction	Cache side-channel	SW	Negligible	Medium
L2 cache way partitioning	Cache side-channel	HW	Area	High
Speculative cache-fill suppress.	All classes	HW	Area/latency	Very High

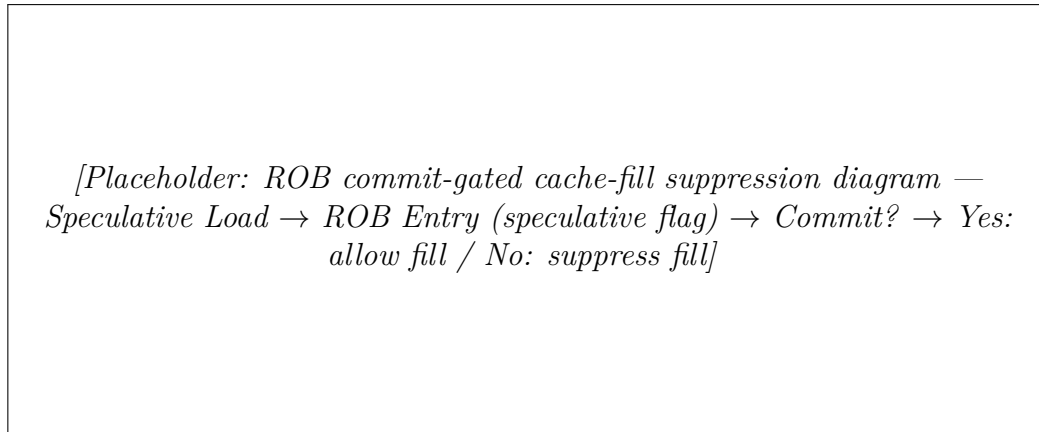


Figure 8.3: Commit-gated cache-fill suppression: cache updates are held until ROB commit, preventing transient execution from producing observable side-effects.

8.1.7 Vulnerability Summary

Table 8.5: Transient Execution Vulnerability Assessment — RWU64IM Multi-Core Extension

Vulnerability	Status	Determining Factor
Meltdown (CVE-2017-5754)	Not applicable	No virtual memory; no page-fault mechanism
Spectre V1 (CVE-2017-5753)	Conditionally present	HW preconditions met; risk conditional on m
Spectre V2 (CVE-2017-5715)	Not applicable	No separate execution contexts; no isolated vi
L1 cache timing channel	Present (per core)	Shared per-core L1; <code>rdcycle</code> accessible
L2 cache timing channel	Present (cross-core)	Shared L2 observable across all cores

The design achieves immunity to Meltdown and Spectre V2 through the *absence of required architectural preconditions* rather than through active mitigation. The primary residual risk is Spectre V1 in multi-task software deployments and the cross-core cache timing channel introduced by the shared L2. Both are addressable by the software and hardware countermeasures described in Section 8.1.6.

Chapter 9

Electrical Characteristics

Table 9.1: Revision History – Chapter 7

Version	Date	Author	Description
1.0	January 22, 2026	Y. Sangale	Initial Draft

This Section will be completed later.

9.1 Operating Conditions

Table 9.2: Operating Conditions

Parameter	Typical	Unit
Supply Voltage	3.3	V
Clock Frequency	80	MHz
Operating Temp Range	-40 to +85	°C

Bibliography

- [1] M. Lipp et al., “Meltdown: Reading Kernel Memory from User Space,” in *27th USENIX Security Symposium*, USENIX Association, 2018, pp. 973–990.
- [2] P. Kocher et al., “Spectre Attacks: Exploiting Speculative Execution,” in *40th IEEE Symposium on Security and Privacy*, IEEE, 2019, pp. 1–19.
- [3] Intel Corporation, “Analysis of Speculative Execution Side Channels,” Intel Corporation, Tech. Rep., 2018, White Paper, Revision 4.0.
- [4] ARM Limited, “Cache Speculation Side-channels,” ARM Limited, Tech. Rep., 2018, Whitepaper, Version 2.4.
- [5] D. Gruss, C. Maurice, A. Fogh, and S. Mangard, “Flush+Flush: A Fast and Stealthy Cache Attack,” in *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, Springer, 2016, pp. 279–299.
- [6] RISC-V International, “RISC-V Cache Management Operation ISA Extensions (Zicbom),” RISC-V International, Tech. Rep., 2022, Version 1.0.
- [7] J. R. Hauser, “TestFloat: Floating-Point Verification,” UC Berkeley, Tech. Rep., 2010, <http://www.jhauser.us/arithmetic/TestFloat.html>.